

TUGAS LAB 1

Implementasi BM25 & Evaluasi NDCG

Mata Kuliah: Information Retrieval

Bobot: 15% dari nilai akhir

Dikerjakan: Individu

Durasi: 2 minggu

Komponen	Keterangan
Tujuan	Memahami BM25 dari prinsip pertama, mengimplementasi dalam Python, dan mengevaluasi hasil ranking menggunakan metrik NDCG
Output	Kode Python (.py) + Laporan analisis (.pdf atau .docx)
Bahasa	Python 3.x (tanpa library rank_bm25 — implementasi manual wajib)
Dataset	Disediakan dalam starter code (mini corpus 8 dokumen)

1. Latar Belakang & Motivasi

BM25 (Best Matching 25) adalah algoritma ranking probabilistik yang dikembangkan pada era 1990-an dan hingga kini menjadi tulang punggung search engine modern seperti Elasticsearch, Apache Solr, dan OpenSearch.

Di era 2026, BM25 semakin relevan karena sistem Retrieval-Augmented Generation (RAG) — yang mendasari aplikasi AI modern — menggunakan BM25 sebagai komponen retrieval utama sebelum proses generasi teks. Memahami BM25 dari prinsip pertama adalah fondasi wajib sebelum mempelajari neural IR.

1.1. Formula BM25

Skor relevansi dokumen D terhadap query Q dihitung dengan:

$$\text{score}(D, Q) = \sum \text{IDF}(q_i) \times [f(q_i, D) \times (k_1 + 1)] / [f(q_i, D) + k_1 \times (1 - b + b \times |D| / \text{avgdl})]$$

Keterangan komponen formula:

- $f(q_i, D)$ = frekuensi term q_i dalam dokumen D (Term Frequency)
- $|D|$ = panjang dokumen D (jumlah token)
- avgdl = rata-rata panjang semua dokumen dalam koleksi
- k_1 = parameter saturasi TF (nilai umum: 1.2 atau 1.5)
- b = parameter normalisasi panjang dokumen (nilai umum: 0.75)
- $\text{IDF}(q_i) = \ln[(N - \text{df}(q_i) + 0.5) / (\text{df}(q_i) + 0.5) + 1] \rightarrow N$ = jumlah total dokumen, df = document frequency

1.2. Metrik Evaluasi: NDCG

Normalized Discounted Cumulative Gain (NDCG) mengukur kualitas hasil ranking dengan mempertimbangkan posisi dokumen relevan. Semakin tinggi posisi dokumen relevan, semakin besar kontribusinya.

$$\text{DCG}@k = \sum \text{rel}(i) / \log_2(i + 1) \quad \text{untuk } i = 1 \text{ sampai } k$$
$$\text{NDCG}@k = \text{DCG}@k / \text{IDCG}@k$$

di mana **IDCG@k** adalah DCG ideal (ranking sempurna berdasarkan relevance label), dan **rel(i)** adalah skor relevansi dokumen di posisi i .

2. Dataset & Struktur Proyek

2.1. Mini Corpus

Anda diberikan sebuah koleksi kecil berisi 8 dokumen tentang topik teknologi (sudah disediakan dalam starter code). Koleksi ini memiliki:

- 8 dokumen teks pendek dalam bahasa Indonesia
- 3 query uji
- Relevance judgments (qrels) untuk setiap pasangan query-dokumen
- Skala relevansi: 0 (tidak relevan), 1 (relevan), 2 (sangat relevan)

2.2. Struktur File

```
lab1_bm25/
├── corpus.py          # Dataset dokumen + qrels (sudah disediakan)
├── bm25.py           # ← File utama yang harus Anda implementasi
├── evaluation.py     # ← File evaluasi yang harus Anda implementasi
├── main.py           # Runner utama (sudah disediakan, jangan diubah)
└── laporan.pdf/docx  # Laporan analisis (dikumpulkan bersama kode)
```

3. Tugas yang Harus Dikerjakan

Tugas ini terdiri dari 4 bagian dengan total 100 poin.

Bagian A — Preprocessing & Indexing

30 Poin

Implementasi fungsi-fungsi berikut dalam `bm25.py`:

1. Fungsi `tokenize(text)`: tokenisasi sederhana (lowercase + split berdasarkan spasi dan tanda baca). Tidak perlu stemming.
2. Fungsi `build_index(corpus)`: bangun inverted index dan hitung statistik koleksi (avgdl, df per term, tf per dokumen).
3. Fungsi `compute_idf(term, N, df)`: hitung IDF menggunakan formula Robertson-Jones:

$$\text{IDF}(t) = \ln[(N - \text{df}(t) + 0.5) / (\text{df}(t) + 0.5) + 1]$$

Catatan Penilaian A:

Penilaian berdasarkan kebenaran inverted index (struktur data), nilai IDF untuk 5 term uji yang akan diberikan saat penilaian, dan ketepatan nilai avgdl.

Bagian B — Implementasi BM25 Scoring

35 Poin

Implementasi fungsi `compute_bm25_score(query, doc_id, index, k1, b)` yang menghitung skor BM25 untuk satu pasang (query, dokumen). Kemudian implementasi fungsi `rank(query, index, k1=1.5, b=0.75)` yang mengembalikan daftar dokumen terurut dari skor tertinggi ke terendah.

Persyaratan implementasi:

- Gunakan nilai default `k1=1.5` dan `b=0.75`
- Hanya dokumen yang mengandung minimal satu term query yang perlu dihitung skornya
- Kembalikan list of tuples: `[(doc_id, score), ...]` terurut descending by score
- Tangani edge case: query term yang tidak ada dalam index (score term = 0)

Larangan Keras:

Dilarang menggunakan library `rank_bm25`, `whoosh`, `elasticsearch-py`, atau library sejenis yang sudah mengimplementasi BM25. Seluruh formula harus ditulis manual. Penggunaan library tersebut akan menghasilkan nilai 0 pada Bagian B.

Bagian C — Evaluasi NDCG

20 Poin

Implementasi dalam `evaluation.py`:

4. Fungsi `compute_dcg(ranked_list, qrels, k)`: hitung DCG@k dari hasil ranking Anda.
5. Fungsi `compute_idcg(qrels, k)`: hitung DCG ideal (IDCG@k) dengan mengurutkan semua dokumen berdasarkan skor relevansi tertinggi.

6. Fungsi `compute_ndcg(ranked_list, qrels, k)`: hitung $NDCG@k = DCG@k / IDC@k$. Kembalikan 0 jika $IDC@k = 0$.
7. Evaluasi tiga nilai NDCG: $NDCG@3$, $NDCG@5$, dan $NDCG@10$ untuk setiap query.

Bagian D — Analisis & Laporan

15 Poin

Tulis laporan singkat (minimal 2 halaman) yang menjawab pertanyaan-pertanyaan berikut:

8. Tampilkan hasil ranking untuk setiap query beserta skor BM25-nya dalam bentuk tabel.
9. Tampilkan nilai $NDCG@3$, $NDCG@5$, $NDCG@10$ untuk setiap query dalam tabel evaluasi.
10. Analisis: Apakah ada query di mana hasil ranking BM25 tidak sesuai intuisi? Mengapa?
11. Eksplorasi parameter: Coba $k_1 \in \{0.5, 1.2, 1.5, 2.0\}$ dengan $b=0.75$ tetap. Bagaimana pengaruhnya terhadap NDCG?
12. Refleksi: Apa kelemahan BM25 yang Anda temukan dari eksperimen ini?

4. Starter Code

4.1. corpus.py (Sudah Disediakan)

Salin kode berikut persis apa adanya ke dalam corpus.py — jangan diubah:

```
# corpus.py - Dataset untuk Lab 1 IR
# Jangan diubah!

CORPUS = {
    "D1": "mesin pencari menggunakan algoritma untuk merangking dokumen
    berdasarkan relevansi query pengguna",
    "D2": "BM25 adalah algoritma ranking probabilistik yang digunakan dalam
    sistem information retrieval modern",
    "D3": "Google dan Bing menggunakan berbagai teknik ranking termasuk machine
    learning dan neural network",
    "D4": "inverted index adalah struktur data utama dalam sistem information
    retrieval untuk pencarian cepat",
    "D5": "TF-IDF mengukur pentingnya sebuah term dalam dokumen relatif
    terhadap seluruh koleksi dokumen",
    "D6": "BERT dan transformer digunakan dalam neural information retrieval
    untuk memahami semantik query",
    "D7": "evaluasi sistem retrieval menggunakan metrik seperti precision
    recall MAP dan NDCG",
    "D8": "query expansion meningkatkan recall dengan menambahkan term sinonim
    ke dalam query original",
}

QUERIES = {
    "Q1": "algoritma ranking dokumen",
    "Q2": "evaluasi information retrieval",
    "Q3": "neural network search engine",
}

# Relevance judgments: {query_id: {doc_id: relevance_score}}
# Skala: 0=tidak relevan, 1=relevan, 2=sangat relevan
QRELS = {
    "Q1": {"D1": 1, "D2": 2, "D3": 0, "D4": 1, "D5": 1, "D6": 0, "D7": 0, "D8":
    0},
    "Q2": {"D1": 0, "D2": 0, "D3": 0, "D4": 0, "D5": 1, "D6": 0, "D7": 2, "D8":
    1},
    "Q3": {"D1": 0, "D2": 0, "D3": 2, "D4": 0, "D5": 0, "D6": 2, "D7": 0, "D8":
    0},
}
```

4.2. bm25.py (Template — Anda Lengkapi)

```
# bm25.py - Implementasi BM25 dari prinsip pertama
# Nama : _____
# NIM : _____

import math
import re
from collections import defaultdict
```

```

def tokenize(text: str) -> list[str]:
    """
    Tokenisasi teks: lowercase, hapus tanda baca, split per spasi.
    Input : string teks
    Output: list of tokens
    """
    # TODO: implementasi Anda di sini
    pass

def build_index(corpus: dict) -> dict:
    """
    Bangun inverted index dan hitung statistik koleksi.
    Input : corpus = {doc_id: teks_dokumen}
    Output: dict berisi:
        index['tf']      : {doc_id: {term: frekuensi}}
        index['df']      : {term: jumlah_dokumen_yang_mengandung_term}
        index['dl']      : {doc_id: panjang_dokumen}
        index['avgdl']   : float, rata-rata panjang dokumen
        index['N']       : int, total jumlah dokumen
        index['docs']    : list of all doc_ids
    """
    # TODO: implementasi Anda di sini
    pass

def compute_idf(term: str, N: int, df: dict) -> float:
    """
    Hitung IDF menggunakan formula Robertson-Jones.
    Formula:  $\ln((N - df[term] + 0.5) / (df[term] + 0.5) + 1)$ 
    Jika term tidak ada dalam df, kembalikan 0.
    """
    # TODO: implementasi Anda di sini
    pass

def compute_bm25_score(query_terms: list, doc_id: str,
                        index: dict, k1: float, b: float) -> float:
    """
    Hitung BM25 score untuk satu pasang (query, dokumen).
    Input : query_terms = list of tokens query
           doc_id       = id dokumen target
           index         = inverted index dari build_index()
           k1, b         = parameter BM25
    Output: float, skor BM25
    """
    # TODO: implementasi Anda di sini
    pass

def rank(query: str, index: dict,
         k1: float = 1.5, b: float = 0.75) -> list:
    """
    Ranking seluruh dokumen untuk query tertentu.
    """

```

```

Input : query = string query
        index = inverted index
Output: list of (doc_id, score) terurut descending by score
"""

# TODO: implementasi Anda di sini
pass

```

4.3. evaluation.py (Template — Anda Lengkapi)

```

# evaluation.py — Metrik evaluasi NDCG
# Nama : _____
# NIM : _____

import math

def compute_dcg(ranked_list: list, qrels: dict, k: int) -> float:
    """
    Hitung DCG@k.
    Input : ranked_list = [(doc_id, score), ...] terurut descending
            qrels        = {doc_id: relevance_score} untuk query ini
            k             = cut-off depth
    Output: float, nilai DCG@k
    """
    # TODO: implementasi Anda di sini
    pass

def compute_idcg(qrels: dict, k: int) -> float:
    """
    Hitung IDCG@k (DCG ideal / perfect ranking).
    Input : qrels = {doc_id: relevance_score}
            k     = cut-off depth
    Output: float, nilai IDCG@k
    """
    # TODO: implementasi Anda di sini
    pass

def compute_ndcg(ranked_list: list, qrels: dict, k: int) -> float:
    """
    Hitung NDCG@k = DCG@k / IDCG@k.
    Kembalikan 0.0 jika IDCG = 0.
    """
    # TODO: implementasi Anda di sini
    pass

```

4.4. main.py (Sudah Disediakan — Jangan Diubah)

```

# main.py — Runner utama Lab 1
from corpus import CORPUS, QUERIES, QRELS
from bm25 import build_index, rank
from evaluation import compute_ndcg

```



```

def main():
    index = build_index(CORPUS)
    print(f'Koleksi: {index["N"]} dokumen | avgdl = {index["avgdl"]:.2f}
token\n')

    for qid, query_text in QUERIES.items():
        print(f'=== {qid}: "{query_text}" ===')
        results = rank(query_text, index)
        qrels = QRELS[qid]

        print(f'  {'Rank':<6} {'Doc':<5} {'BM25 Score':<14} {'Relevansi'}')
        print(f'  {"-"*40}')
        for i, (doc_id, score) in enumerate(results, 1):
            rel = qrels.get(doc_id, 0)
            print(f'  {i:<6} {doc_id:<5} {score:<14.4f} {rel}')

        for k in [3, 5, 10]:
            ndcg = compute_ndcg(results, qrels, k)
            print(f'  NDCG@{k} = {ndcg:.4f}')
        print()

if __name__ == '__main__':
    main()

```

5. Kriteria Penilaian

Bagian	Kriteria	Poin
A. Preprocessing & Index	tokenize() benar, inverted index akurat, IDF formula tepat, avgdl benar	30
B. BM25 Scoring	Formula BM25 lengkap dan benar, hasil ranking masuk akal untuk semua query, edge case ditangani	35
C. NDCG Evaluation	DCG, IDCG, NDCG dihitung dengan benar, nilai NDCG@3/5/10 akurat	20
D. Laporan Analisis	Tabel hasil disajikan rapi, analisis parameter k1 ada, refleksi kelemahan BM25 insightful	15
TOTAL		100

5.1. Cara Pengujian Otomatis

Dosen akan menjalankan main.py dengan corpus yang sama dan membandingkan output dengan nilai referensi. Pastikan program berjalan tanpa error dan output sesuai format yang dicontohkan di main.py.

5.2. Bonus

- (+5 poin) Implementasi preprocessing lebih baik: stopwords removal menggunakan list manual.
- (+5 poin) Visualisasi distribusi skor BM25 dalam bentuk bar chart menggunakan matplotlib.
- (+10 poin) Implementasi BM25+ (varian BM25 yang mengatasi masalah zero lower-bound). Bandingkan NDCG-nya.

6. Petunjuk Teknis & Tips

6.1. Cara Memulai

13. Buat folder lab1_bm25/ dan copy semua file template.
14. Mulai dari tokenize() — pastikan benar dulu sebelum lanjut.
15. Test build_index() secara manual: print isi index['tf'] dan index['df'] untuk memverifikasi.
16. Hitung BM25 untuk satu pasang (Q1, D2) secara manual di kertas, lalu bandingkan dengan output kode Anda.
17. Terakhir baru implementasi NDCG.

6.2. Contoh Perhitungan Manual (Untuk Verifikasi)

Coba hitung manual BM25("algoritma", D2) dengan $k_1=1.5$, $b=0.75$. Langkah-langkah:

- Hitung $tf("algoritma", D2)$: berapa kali kata ini muncul di D2?
- Hitung $df("algoritma")$: di berapa dokumen kata ini muncul?
- Hitung IDF: $\ln((8 - df + 0.5)/(df + 0.5) + 1)$
- Hitung $|D2|$ dan $avgdl$ dari seluruh corpus
- Substitusi ke formula BM25

Gunakan hasil ini sebagai ground truth untuk memverifikasi implementasi Anda.

6.3. FAQ

18. Bolehkah pakai numpy? Ya, boleh untuk operasi matematika dasar.
19. Apakah harus ada GUI? Tidak. Output terminal sudah cukup.
20. Bagaimana jika corpus mengandung angka? Cukup perlakukan sebagai token biasa.
21. Apakah perlu stemming? Tidak wajib, namun dapat dilakukan sebagai bonus.

6.4. Pengumpulan

Kumpulkan satu file ZIP bernama NIM_Lab1IR.zip berisi:

- bm25.py — implementasi BM25
- evaluation.py — implementasi NDCG
- laporan.pdf atau laporan.docx — laporan analisis
- corpus.py dan main.py — tidak perlu diubah, tapi tetap disertakan

Selamat mengerjakan! Baca seluruh soal sebelum mulai coding.